

Travaux Pratiques 1 – Introduction aux Réseaux de Neurones et CNN

Exercice 1 – Neurone artificiel

On considère un neurone artificiel défini par :

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + b, y = f(z)$$

Données :

$$x_1 = 1, x_2 = 2$$

$$w_1 = 0.5, w_2 = -1$$

$$b = 0.5$$

Questions :

1. Calculer z
2. Calculer y avec la fonction seuil
3. Calculer y avec la fonction ReLU
4. Refaire avec b = -1 et commenter

Exercice 2 – Perceptron et séparabilité linéaire

Considérer les données suivantes :

x1	x2	y
1	1	1
2	1	1
1	2	1
-1	-1	0
-2	-1	0
-1	-2	0

1. Tracer les points
2. Dire s'ils sont séparables linéairement
3. Le perceptron peut-il apprendre ?
4. Tester le problème XOR et conclure

Exercice 3 – MLP conceptuel

MLP avec 2 entrées, 1 couche cachée (2 neurones), 1 sortie.

Jeu de données :

Échantillon 1 : $x_1 = 0, x_2 = 1$, cible = 1

Échantillon 2 : $x_1 = 1, x_2 = 0$, cible = 1

Fonction d'activation : Sigmoid

$$\text{sigmoid}(x) = 1 / (1 + e^{(-x)})$$

$$\text{sigmoid}'(x) = y(1 - y)$$

Taux d'apprentissage : $\eta = 0,5$

Poids initiaux et biais :

$$w_1 (x_1 \rightarrow h_1) = 0,10$$

$$w_2 (x_2 \rightarrow h_1) = 0,20$$

$$w_3 (x_1 \rightarrow h_2) = 0,30$$

$$w_4 (x_2 \rightarrow h_2) = 0,40$$

$b1 = 0,10$

$b2 = 0,10$

$w5 (h1 \rightarrow h3) = 0,50$

$w6 (h2 \rightarrow h3) = 0,60$

$b3 = 0,10$

Fonction de perte :

Erreur quadratique moyenne (MSE) :

$L = 1/2 (\text{cible} - y)^2$

Le facteur 1/2 est ajouté intentionnellement pour simplifier la dérivée par la suite.

1. Écrire la propagation avant
2. Expliquer la rétropropagation
3. Pourquoi une activation non linéaire ?

Exercice 4 – MLP avec scikit-learn

Dataset utilisé : Iris dataset

Lien officiel (UCI) :

<https://archive.ics.uci.edu/ml/datasets/iris>

Disponible aussi directement dans :

scikit-learn via `load_iris()`

- ✓ 150 échantillons
- ✓ 4 variables numériques :
 - longueur sépale
 - largeur sépale
 - longueur pétale
 - largeur pétale
- ✓ 3 classes :
 - Setosa
 - Versicolor
 - Virginica
- 1. Charger le dataset
- 2. Séparer les données en train/test
- 3. Normaliser les variables
- 4. Définir un MLP avec :
 - 1 couche cachée
 - 5 neurones
- 5. Entraîner le modèle
- 6. Mesurer :
 - Accuracy train
 - Accuracy test
- 7. Analyser les résultats
 - Pourquoi faut-il normaliser les données avant un MLP ?
 - Combien d'entrées possède le réseau ?
 - Combien de paramètres approximativement ?
 - Le modèle converge-t-il rapidement ?
 - Y a-t-il du sous-apprentissage ?
 - Le modèle généralise-t-il correctement ?
- 8. Reprendre exactement le même protocole
 - Modifier uniquement la taille de la couche cachée (50 neurones)
 - Comparer : Temps d'entraînement, accuracy train, accuracy test

Exercice 5 – Images avec MLP (MNIST)

Dataset utilisé : MNIST

Lien officiel :

<http://yann.lecun.com/exdb/mnist/>

Disponible via :

- `fetch_openml("mnist_784")` dans scikit-learn

- ✓ 70 000 images
- ✓ Images en niveaux de gris
- ✓ Taille : 28×28 pixels
- ✓ 10 classes (chiffres 0–9)
- ✓ Chaque pixel $\in [0,255]$

1. Charger le dataset
2. Séparer train/test
3. Normaliser les pixels (ex: division par 255)
4. Aplatir les images : Transformer chaque image 28×28 en vecteur de 784 valeurs
5. Définir un MLP (ex : 1 couche cachée)
6. Choisir :
 - Nombre de neurones
 - Nombre d'itérations
 - Fonction d'activation
7. Entraîner le modèle
8. Mesurer :
 - Accuracy train
 - Accuracy test
 - Temps d'entraînement